Chapter 44 - Debugging and Profiling Cookbook

IE Mon is the workbench. IE Script is the repeatable notebook. This chapter shows common tasks rather than another full command reference.

Use Chapter 33 when you need the command table. Use Chapter 34 when you need the full script module list.

44.1 Stop At A Known Address

The smallest debugging loop is still:

```
(ie64)> d 1000 #4
(ie64)> b 1018
(ie64)> r pc 1000
(ie64)> g
(ie64)> r
(ie64)> bc 1018
```

`d` proves the bytes. `b` sets a breakpoint. `r pc 1000` sets the entry point. `g` runs until the breakpoint. The final `r` shows what changed.

If the programme runs past the place you expected, set the breakpoint on the self-loop or on the first MMIO write after the setup sequence.

44.2 Catch A Bad MMIO Write

Use a write watchpoint when you know the exact address:

```
(ie64)> bpmdw F0044
(ie64)> g
(ie64)> wl
(ie64)> wc F0044
```

`bpmdw` watches a 32-bit write. The monitor also has byte, word, and quad forms:

Command	Width	Access
`bpmbw addr`	byte	write
`bpmww addr`	word	write
`bpmdw addr`	long	write
`bpmqw addr`	quad	write
`bpmdr addr`	long	read
`bpmd addr`	long	read or write

Use `wl` to list watchpoints and `wc addr` to clear one.

44.3 Find Who Touched A Region

When you do not know the exact access, turn on the access log:

```
(ie64)> accesslog on #64
(ie64)> g
(ie64)> accesslog show #10
(ie64)> who wrote F0044
(ie64)> accesslog off
```

`accesslog` records recent reads, writes, and instruction fetches. `who wrote addr` asks for the last recorded writer of one address.

Some access tools require CPU and bus access instrumentation. If the monitor says the service is unavailable, fall back to a normal watchpoint or a smaller breakpoint-driven session.

44.4 Use I/O Views

The `io` command reads a named device view. It is often clearer than a raw memory dump:

```
(ie64)> io video
(ie64)> io blitter
(ie64)> io voodoo
(ie64)> io audio
(ie64)> io coproc
(ie64)> io midilive
```

Use I/O views after a setup routine and before running the next part of the programme. They answer the basic question: did the hardware receive the state I meant to write?

44.5 Listen While CPUs Are Stopped

Entering IE Mon freezes both the CPUs and the audio clock. Inspect the held state first, then use `ta` when you need to hear what the stopped programme configured:

```
(ie64)> io audio
(ie64)> ta
(ie64)> fa
(ie64)> io audio
```

While thawed, players and sound engines advance even though the CPUs remain stopped. `fa` holds them again for a stable second inspection. An explicit `fa` or `ta` becomes the state that remains after monitor exit. Finish with `ta` before `x` when sound should continue, or with `fa` when it should remain frozen.

44.6 Symbols

Symbols let the monitor use names in address expressions:

```
(ie64)> sym add loop 1018
(ie64)> sym lookup loop
(ie64)> b loop
(ie64)> d loop #2
```

You can also resolve an address:

```
(ie64)> sym resolve 1018
```

For PRG-style work, `sym add` is usually enough. The monitor also knows how to load label and ELF symbol files, but the reader path in this guide does not require those files.

44.7 Reverse Step

`bs` or `rs` steps backwards through the CPU-local timeline. Use `history horizon` when you want to inspect the whole-machine reverse snapshot horizon.

```
(ie64)> g
(ie64)> bs
(ie64)> r
(ie64)> history horizon
```

Reverse history is a debugging aid. Do not use it as a timing device. If the bug depends on an interrupt or a VBlank edge, record the device status registers as well as the CPU registers.

44.8 Automate A Repro With IE Script

This script types a short BASIC programme, waits for output, then asks the video module for a frame hash.

```
term.type_line('10 SCREEN 1')
term.type_line('20 PALETTE 1,255,0,0')
term.type_line('30 PLOT 10,10,1')
term.type_line('RUN')
term.wait_output('Ready', 2000)
sys.wait_frames(2)
sys.print(video.frame_hash())
```

Use this pattern when a bug needs the same setup every time. The script does not replace the programme. It presses the keys, waits for the machine, and records the observation.

44.9 Capture Output

For terminal repros:

```
sys.capture_output('run.log')
term.type_line('RUN')
term.wait_output('Ready', 2000)
sys.capture_output_off()
```

For ordinary video repros, prefer a stable frame hash before saving a screenshot:

```
sys.wait_frames(3)
h = video.frame_hash()
sys.print(h)
rec.screenshot('frame.png')
```

The hash is the quick comparison. The screenshot is the human check. A frame hash describes the guest picture and is the right first test for drawing, palette, blitter, and compositor faults.

When the composed picture is correct but the displayed picture is not, capture the two presentation stages separately:

```
old_mode = video.get_crt_mode()
video.set_crt_mode('curved')
sys.wait_frames(2)
rec.screenshot_composed('before.png')
rec.screenshot_screen('after.png')
video.set_crt_mode(old_mode)
```

`before.png` is the composition before CRT processing, cursor, and status bar. `after.png` is the final displayed frame. If the fault appears in both, begin with the video card, blitter, or compositor. If it appears only in `after.png`, investigate final presentation. These calls raise a script error when the selected output cannot provide the requested control or capture stage.

44.9.1 Check presentation scale

If a picture has the right pixels but the wrong proportions, compare the two presentation-scale modes before changing guest drawing code:

```
old_scale = video.get_scale_mode()

video.set_scale_mode('fit')
sys.wait_frames(2)
rec.screenshot_composed('fit.png')

video.set_scale_mode('stretch')
sys.wait_frames(2)
rec.screenshot_composed('stretch.png')

video.set_scale_mode(old_scale)
```

If `fit.png` has the intended proportions and `stretch.png` does not, the source geometry is sound and presentation stretching caused the difference. If both are wrong, inspect the display-card mode, framebuffer dimensions, and drawing calculations. This comparison changes no guest register or source framebuffer byte, and restores the scale mode that was active before the test.

44.10 Inspect A Stopped Z80 Load

When a Z80 programme needs register setup before its first instruction, load it while stopped, set the register, then start it:

```
-- INC A; HALT
sys.write_file('PROBE.IE80', string.char(0x3C, 0x76))
cpu.load_stopped('PROBE.IE80')
dbg.set_reg('A', 41)
sys.print('STOPPED ' .. tostring(not cpu.is_running()))

cpu.start()
sys.wait_ms(10)
cpu.stop()
sys.print('A ' .. dbg.get_reg('A'))
```

Run this with Z80 selected. The stored image is checked and loaded before execution begins. `STOPPED true` proves that no instruction ran during loading. The script then sets `A` to `41`; `INC A` changes it to `42`, and `HALT` prevents a second increment. The final line is `A 42`. This pattern is useful when a fault depends on initial register or memory state that must exist before the first instruction.

44.11 Use Performance Reports Carefully

`sys.perf_reset()` and `sys.perf_report()` measure instrumented subsystems when performance accounting is active:

```
sys.perf_reset()
sys.wait_frames(60)
report = sys.perf_report()
sys.print(report)
```

An empty report means either performance accounting is off or no instrumented path ran during the measured span. A non-empty report is a guide to where time went. It is not a promise that the same programme will take the same time on every machine.

44.12 Measure IE64 And x86 Execution

`sys.perf_report()` measures time in instrumented machine subsystems. `cpu.jit_stats()` answers a different question: how the selected CPU executed its programme. For IE64 and x86, compare two snapshots around a representative interval:

```
local before = cpu.jit_stats()
sys.wait_frames(60)
local after = cpu.jit_stats()

local function change(name)
  return (after[name] or 0) - (before[name] or 0)
end

sys.print("NATIVE " .. change("native_retired"))
sys.print("FALLBACK " .. change("fallback_instructions"))
sys.print("INVALIDATIONS " .. change("invalidations"))
sys.print("CACHE HITS " .. change("cache_hits"))
sys.print("CACHE MISSES " .. change("cache_misses"))
```

Lines 1 to 3 bracket sixty completed frames without resetting the running programme. Lines 5 to 7 turn cumulative counters into interval counts. The final lines report generated-code retirement, fallback execution, code invalidation, and cache lookup results.

If `native_retired` remains zero, check `native_entries` and `cpu.execution_mode()` before concluding that generated code ran. A high fallback count points to work that the selected backend did not complete directly. Repeated invalidation suggests that the programme is rewriting code. Many cache misses beside few hits suggest poor reuse. Region candidates without corresponding compiled regions show attempted promotion that did not install a region. Large helper or I/O counts show frequent hand-offs for individual operations.

Read these as comparisons between equivalent runs. They are cumulative diagnostic counts, not timing guarantees. Use `sys.perf_report()` beside them when the question is which video, audio, bus, or Voodoo stage consumed time.

44.13 A Practical Debug Order

When a programme misbehaves, use this order:

1. `d` the code or `LIST` the BASIC programme.
2. Set one breakpoint at the first wrong-looking step.
3. Dump the memory or I/O view before and after the step.
4. Add one watchpoint only if the writer is unknown.
5. Use `accesslog` or `who` only when watchpoints are not enough.
6. Check `history horizon` if reverse snapshots matter to the fault.
7. Turn the session into an IE Script only after the manual steps are understood.

The fewer moving parts in the debug session, the more likely the answer is the real machine state.